AVR CONSOLE

Frontend UX/UI Design & Implementation Specification

Project: AVR-Map
Problem Statement: SIH26053
Frontend Product Name: AVR Console
Version: 1.0
Purpose: SIH Prototype Operator & Demonstration Interface

---

1. Product Definition

AVR Console is the visual interface for AVR-Map.

It is not merely a dashboard.

It serves three simultaneous purposes:

Engineering interface

Allows developers to inspect perception, mapping, tracking and system performance.

Operator interface

Allows someone to understand the current environment and health of AVR-Map without reading ROS topics or terminal logs.

SIH demonstration interface

Makes the core AVR-Map innovation immediately visible to judges.

The interface must communicate:

```
WHAT DOES THE VEHICLE SEE?

WHAT DOES IT UNDERSTAND?

HOW DETAILED IS THE MAP HERE?

WHY DID THIS REGION RECEIVE THAT RESOLUTION?

IS THE TERRAIN SAFE?

WHAT IS MOVING?
```

```
HOW CERTAIN IS THE SYSTEM?

IS ADAPTIVE MAPPING ACTUALLY SAVING COMPUTE/MEMORY?
```

---

2. Main UX Objective

A judge looking at AVR Console for the first time should understand within approximately 30–60 seconds:

```
Raw LiDAR
    ↓
Semantic understanding
    ↓
Adaptive 2.5D map
    ↓
Different spatial resolutions
    ↓
Important objects receive finer detail
    ↓
Less-important distant terrain stays coarse
    ↓
Memory and computation are reduced
```

without requiring a long verbal explanation.

---

3. Design Philosophy

The UI will follow this principle:

> **The scene is the interface.**

The LiDAR/map visualization occupies most of the available screen.

Controls, diagnostics and metrics support the scene.

They do not dominate it.

This is different from generic dashboards where the screen is filled with cards containing disconnected statistics.

---

4. Visual Concept

The visual system shall be called:

Adaptive Instrument UI

Its characteristics:

```
Precise
Dense but ordered
Technical
Quiet
Purposeful
Data-driven
Dark-first
Minimal decoration
Strong hierarchy
Low distraction
```

It should feel closer to:

```
scientific equipment
+
robotics workstation
+
flight instrumentation
+
spatial-analysis software
```

than:

```
startup dashboard
crypto dashboard
AI chatbot
gaming HUD
cyberpunk interface
```

---

5. What We Explicitly Avoid

Do NOT generate:

```
huge gradient backgrounds

purple/blue AI gradients

glassmorphic cards everywhere

rounded-2xl dashboard cards

floating translucent blobs

Bento-grid marketing layouts

excessive pills

sparkle/AI icons

giant marketing headings

neon glows around every element

fake radar circles

random hexagons

animated background particles

fake terminal text

3D decorative objects unrelated to data
```

The LiDAR points themselves already provide the visual complexity.

The UI chrome should remain restrained.

---

6. Core Visual Motif

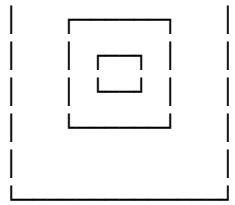
AVR-Map is based on hierarchical map cells.

Therefore the visual identity should use:

Nested Squares

Concept:

```
┌──────────────┐
│              │
```



This represents:

```
40 cm
 ↓
20 cm
 ↓
10 cm
 ↓
5 cm
```

It can become:

- AVR logo mark;
- loading indicator;
- resolution legend;
- selected-cell indicator;
- favicon;
- empty-state motif.

Use it sparingly.

---

7. Wordmark

Recommended:

```
▣ AVR/MAP
   ADAPTIVE 2.5D PERCEPTION
```

or:

```
▣ AVR MAP
  2.5D
```

Avoid futuristic typography.

The product should appear like a serious engineering tool.

---

8. Information Architecture

Primary sections:

```
MISSION

COMPARE

BENCHMARKS

DIAGNOSTICS

CONFIG
```

No traditional dashboard home page is necessary.

Opening the application should take the user directly to:

```
MISSION
```

because the live spatial representation is the product.

---

9. Application Structure

```
AVR Console
│
├── Mission
│   ├── 3D scene
│   ├── layers
│   ├── object tracks
│   ├── cell inspection
│   └── telemetry
│
├── Compare
│   ├── Uniform map
│   ├── Adaptive map
│   ├── synchronized camera
│   └── performance delta
│
├── Benchmarks
│   ├── experiments
│   ├── memory
│   ├── latency
│   ├── cell count
│   ├── preservation metrics
```

```
│   └── ablations
│
├── Diagnostics
│   ├── pipeline state
│   ├── node latency
│   ├── queues
│   ├── connection health
│   └── logs
│
└── Config
    ├── mapping
    ├── resolution
    ├── semantics
    ├── tracker
    ├── vehicle
    └── visualisation
```

---

10. Desktop Target

AVR Console is a workstation application.

Primary design viewport:

```
1920 × 1080
```

Must also work well at:

```
1600 × 900
1440 × 900
1366 × 768
1280 × 720
```

The SIH demo may run on laptops or projectors, so 1366×768 support is particularly important.

---

11. Mobile Strategy

Do NOT compromise the workstation UI to achieve full mobile control.

Below approximately:

```
1100 px width
```

switch to a simplified monitoring layout.

Mobile may expose:

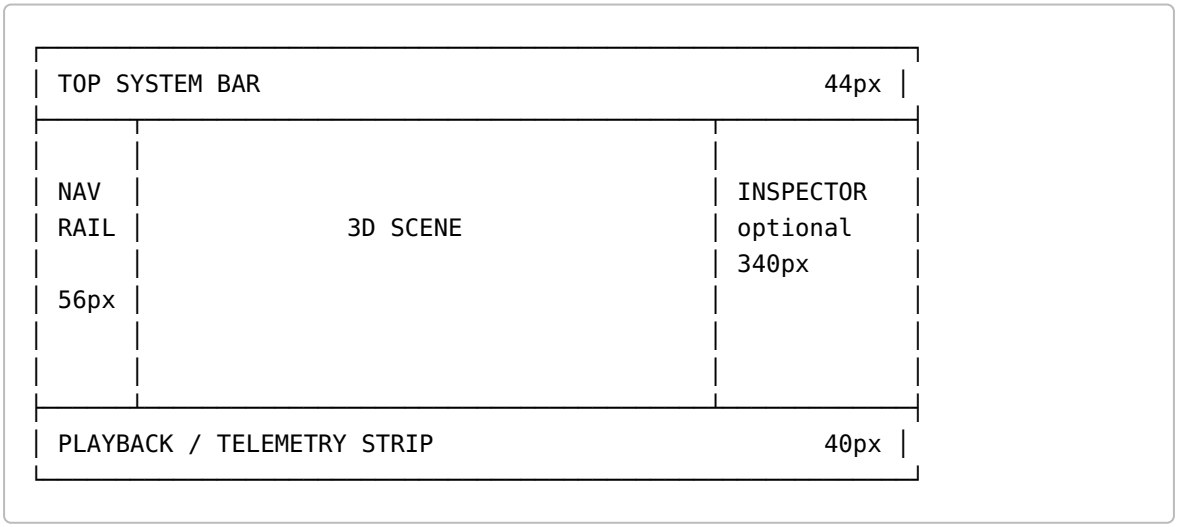
```
system health
FPS
memory
active tracks
current scene snapshot
```

but full 3D engineering controls are not a V1 priority.

---

12. Global Layout

Recommended workstation shell:



---

13. Top System Bar

Height:

```
44 px
```

Contains:

Left

```
AVR logo
AVR/MAP
```

Center-left

Current source:

```
RELLIS-3D / SEQ 03
```

or:

```
LIVE / OUSTER OS1
```

Center

Current mapping mode:

```
FULL ADAPTIVE
```

Right

```
10.0 Hz
p95 74 ms
0 dropped

[ LIVE ]
```

Do not turn each value into a large card.

Use compact text and separators.

Example:

```
▣ AVR/MAP    RELLIS-3D / 03       FULL ADAPTIVE

10.0 Hz  |  p95 74ms  |  DROP 0          ● LIVE
```

---

14. Navigation Rail

Width:

```
56 px
```

Icons vertically centered.

Routes:

```
Mission
Compare
Benchmarks
Diagnostics

————————

Config
```

Use icon-only navigation by default.

Hover reveals labels.

Active item:

```
2px accent bar
+
slightly brighter background
```

Do NOT use glowing icons.

---

15. Icon System

Recommended:

Phosphor Icons

Visual style:

```
regular / medium
16–18 px
```

Reasons:

- less generic than default Lucide-heavy AI interfaces;
- consistent stroke system;
- supports multiple weights;
- technical but not futuristic.

Create custom SVGs only for:

```
adaptive hierarchy
cell resolution
LiDAR
overhang
```

where generic icons are insufficient.

---

16. MISSION Screen

This is the most important screen.

Approximately:

```
70–85% of visible area
```

should belong to the scene.

---

17. Mission Layout

```
┌──────────────────────────────────────────────────────┐
│ Semantic | Resolution | Elevation | Traversability   │
│ Uncertainty | Dynamics                         LIVE  │
│                                                      │
│                                                      │
│                                                      │
│                    3D VIEW                           │
│                                                      │
│                                                      │
│                                                      │
│  LEGEND                            CAMERA CONTROLS   │
│                                                      │
├──────────────────────────────────────────────────────┤
│ FPS | latency | cells | memory | tracks | timeline   │
└──────────────────────────────────────────────────────┘
```

---

18. View Modes

Only one primary colouring mode is active at a time.

Modes:

```
SEMANTIC

RESOLUTION

ELEVATION

TRAVERSABILITY

UNCERTAINTY

DYNAMICS
```

These should look like compact instrument tabs.

Not rounded pills.

Example:

```
SEMANTIC   RESOLUTION   ELEVATION   TRAVERSABILITY
```

The underline marks the active mode.

---

19. Scene Layer Model

Primary mode controls colour.

Independent overlays control visibility.

Example:

```
Primary:
RESOLUTION

Layers:
☑ raw points
☑ adaptive cells
☑ vehicle
☑ tracks
☐ bounding boxes
☑ trajectory
☐ coordinate frames
```

This prevents creating dozens of mutually confusing visual modes.

---

20. Layer Drawer

Layer configuration should be collapsed by default.

Activation:

```
L
```

or layer icon.

Drawer width:

```
260 px
```

Groups:

```
POINT CLOUD

MAP

SEMANTICS

DYNAMICS

DEBUG
```

Use checkboxes and compact sliders.

---

21. Scene Camera Modes

Required:

```
Perspective

Top

Follow Vehicle

Free
```

Controls in bottom-right corner.

Keyboard:

```
1 → Perspective

2 → Top

3 → Follow

R → Reset camera
```

---

22. Perspective Camera

Default SIH presentation view:

```
slightly elevated
behind and above vehicle
```

This makes:

```
LiDAR
terrain
map cells
tracked objects
```

simultaneously understandable.

---

23. Top View

Useful for:

```
adaptive-grid structure
resolution zones
comparison
traversability
```

The top view should preserve metric scale.

---

24. Scene Background

Do not use pure black.

Recommended:

```
#070A0D
```

or perceptual equivalent.

Grid plane:

```
1 m minor grid
5 m major grid
```

Very low contrast.

Grid should disappear gradually with distance.

---

25. World Grid

Grid lines must communicate scale, not decoration.

Recommended:

```
minor:
~5% visual contrast

major:
~10–12%
```

Axis:

```
X → subtle red-grey

Y → subtle green-grey

Z → subtle blue-grey
```

Only if coordinate axes are enabled.

---

26. Point Cloud Rendering

Raw LiDAR points should be rendered using GPU buffers.

Recommended frontend representation:

```
THREE.Points

+

BufferGeometry
```

Three.js specifically supports point-cloud rendering via `Points`, while `BufferGeometry` stores positions/colours in GPU-friendly buffers rather than creating one JS object for every point.

Never render:

```
100,000 React <div>s
```

or:

```
one React component per point
```

---

27. Adaptive Cell Rendering

Cells should be rendered with:

```
THREE.InstancedMesh
```

Group cells by resolution level.

Example:

```
InstancedMesh L0 → all 5cm cells

InstancedMesh L1 → all 10cm cells

InstancedMesh L2 → all 20cm cells

InstancedMesh L3 → all 40cm cells
```

Three.js `InstancedMesh` is specifically intended to reduce draw calls when large numbers of objects use the same geometry/material.

This is significantly better than individual meshes.

---

28. Important Rendering Rule

React controls the interface.

React should **not control every point or map cell**.

Use:

```
React
→ lifecycle / controls

Three.js
→ high-frequency scene state
```

Point/cell arrays should remain in:

```
typed arrays
GPU buffers
refs
```

rather than triggering React component renders.

---

29. Renderer Architecture

```
WebSocket
   ↓
Worker / Decoder
   ↓
Typed Arrays
   ↓
Renderer State
   ↓
Three.js Buffers
   ↓
GPU
```

React state receives only:

```
selected cell
selected track
active mode
metrics
UI state
```

---

30. Frame Rendering vs Data Update

The scene may render:

```
60 FPS
```

while data arrives:

```
10 Hz
```

These are different concepts.

Do not rebuild the full React tree ten times per second.

---

31. Device Pixel Ratio

Rendering a full 3D point cloud at 4K using:

```
devicePixelRatio = 2
```

can be unnecessarily expensive.

Recommended cap:

```
min(devicePixelRatio, 1.5)
```

Performance mode may use:

```
1.0
```

---

32. WebGL vs WebGPU

Use:

WebGL2 for V1

WebGPU is powerful but MDN still classifies it as not universally baseline because browser support remains incomplete.

Therefore:

```
V1:
Three.js WebGL

V2:
experimental WebGPU renderer
```

---

33. Resolution View

This is one of the signature AVR Console views.

Each cell is shown at its true map-cell size.

Colours:

```
5 cm  → turquoise

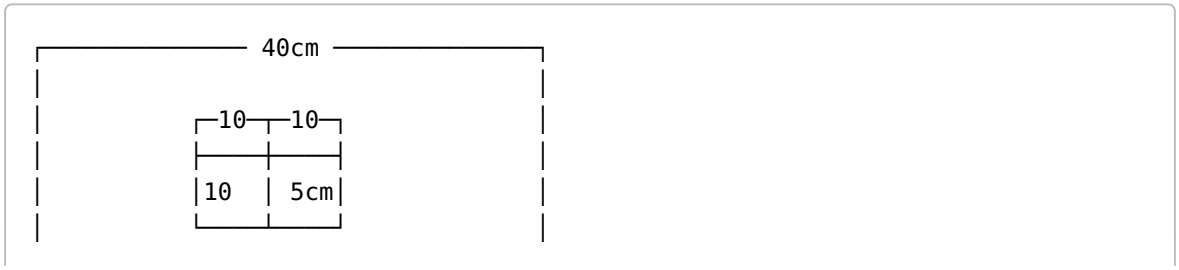
10 cm → blue

20 cm → amber

40 cm → violet
```

A fine object embedded inside coarse terrain should be immediately visible.

Example:



```
│                                    │
└────────────────────────────────────┘
```

---

34. Resolution Legend

Do not use four equal coloured squares.

Instead show actual hierarchy:

```
5cm   ▪

10cm  ■

20cm  ██

40cm  ████
```

or nested rectangles.

This reinforces the concept physically.

---

35. Unique UX Feature — “Why This Resolution?”

When a user selects a map cell, the inspector should explain exactly why the cell has its current resolution.

Example:

```
CELL 0x1AE92

FINAL
10 cm

BASE
40 cm
distance 72.4 m

REFINEMENT

semantic
POLE
-2 levels

uncertainty
```

```
0.16
no refinement

dynamic
false

geometry
stable

density
PASS
18 pts / parent
```

This is one of the strongest interface features in the entire project.

It converts AVR-Map from a black box into an explainable mapping system.

---

36. Cell Inspector

Width:

```
340 px
```

Appears only when:

```
cell selected
```

Sections:

```
IDENTITY

RESOLUTION

GEOMETRY

SEMANTICS

SAFETY

TEMPORAL
```

---

37. Cell Inspector Content

Example:

```
CELL
L1 / 10 cm

POSITION
x  18.320
y  -4.410

GROUND
0.14 m
σ² 0.003

MAX Z
2.31 m

CLEARANCE
2.17 m

SEMANTIC
pole
0.91

HAZARD
pole
0.91

UNCERTAINTY
0.09

TRAVERSABILITY
BLOCKED

POINTS
23

LAST OBSERVED
84 ms
```

Numeric values use mono typography.

---

38. Selection Interaction

Do not continuously raycast every mouse movement over hundreds of thousands of cells.

Use:

```
click-to-inspect
```

rather than expensive continuous hover inspection.

Hover may highlight:

```
coarse tile
```

only when inexpensive.

---

39. Selected Cell Styling

Selected cell:

```
white or pale-cyan outline
2 px screen-space equivalent
```

Neighbouring cells remain unchanged.

Avoid giant glows.

---

40. Track Inspector

Selecting a tracked object replaces the cell inspector.

Example:

```
TRACK 024

PERSON

MOVING

POSITION
x 12.4
y 3.8

VELOCITY
1.26 m/s
```

```
AGE
4.2 s

CONFIDENCE
0.94

RESOLUTION
5 cm enforced

LAST OBSERVED
41 ms
```

---

41. Dynamic Objects

Tracked objects should use:

```
thin bounding volume

track ID

velocity vector
```

Do not fill giant transparent boxes around every vehicle.

Labels are shown for:

```
selected object
nearest N relevant objects
high-risk objects
```

not all tracked objects simultaneously.

---

42. Timeline

Dataset playback requires a bottom timeline.

Example:

```
◂  ▸  1×      00:14.220 ──────●────────── 02:11.000
```

Features:

```
play/pause

speed
scrubbing

frame step

timestamp
```

Keyboard:

```
Space → pause/play

← → → frame step when paused
```

---

43. Live Mode

In live sensor mode, timeline becomes:

```
● LIVE        00:21:18 uptime
```

and may show:

```
latest frame age
```

---

44. Telemetry Strip

Bottom persistent strip:

```
FPS         10.0

E2E         73 ms

MAP         14 ms

CELLS       182k

MEM         24.8 MB

TRACKS      7
```

```
DROP       0
```

No giant metric cards.

Values use tabular numerals.

---

45. COMPARE Screen

This screen should become the **SIH demo killer feature**.

It must compare:

```
UNIFORM
vs
ADAPTIVE
```

on exactly the same frame.

---

46. Compare UX

Provide three comparison modes:

Split

```
UNIFORM | ADAPTIVE
```

two equal synchronized views.

Wipe

One scene over another with draggable divider.

```
← UNIFORM | ADAPTIVE →
```

This should be the default presentation mode.

Difference

Highlight only regions where adaptive resolution differs from the uniform representation.

---

47. Synchronized Camera

In Compare:

```
camera rotation

zoom

position

timestamp
```

must always remain synchronized.

A judge should never wonder whether differences come from different viewpoints.

---

48. Compare Metric Rail

Bottom:

```
UNIFORM                  ADAPTIVE

12.6M     CELLS           0.74M

351MB     MAP MEM         22MB

31ms      UPDATE          13ms

100%      SMALL OBJ       97%
```

Actual measured values only.

---

49. Delta Presentation

Show:

```
-78.6% cells

-63.1% map memory

-41.2% mapping latency
```

but avoid:

```
+500% AMAZING
```

Keep language scientific.

---

50. BENCHMARK Screen

This screen is for evidence.

Not marketing.

Layout:

```
┌──────────────┬──────────────────────────────┐
│ EXPERIMENTS  │                              │
│              │         CHART AREA           │
│ Exp 01       │                              │
│ Exp 02       │                              │
│ Exp 03       ├──────────────────────────────┤
│              │ METRICS / TABLE              │
└──────────────┴──────────────────────────────┘
```

---

51. Benchmark Experiments

Recommended navigation:

```
Adaptive vs Uniform

Distance Ablation

Semantic Refinement

Uncertainty Refinement

Small Objects

Overhang

Dynamic Objects

Rough Terrain
```

---

52. Chart Library

Use:

uPlot

for high-frequency metric plots.

It is designed specifically as a small, memory-efficient streaming time-series library and deliberately avoids unnecessary chart animations.

Avoid using Recharts for real-time system telemetry.

---

53. Chart Style

Charts should look like engineering plots.

```
thin grid

1.5px lines

no gradient fills by default

no rounded chart bars

no 3D charts

no animated entrances

no glowing lines
```

Use crosshair cursor.

Show exact values.

---

54. Chart Colours

Telemetry should reuse system semantic colours.

Example:

```
end-to-end → blue

semantic → violet

mapping → turquoise

tracking → amber

traversability → green
```

Do not assign random colours per chart.

---

55. DIAGNOSTICS Screen

The main visualization should be the pipeline itself.

```
INPUT
  │ 2ms
  ▼
PREPROCESS
  │ 7ms
  ▼
SEMANTIC
  │ 38ms
  ▼
MAP
  │ 14ms
  ├──── TRACK 6ms
  │
  ▼
TRAVERSABILITY
  │ 8ms
  ▼
PUBLISH
```

---

56. Pipeline Health UX

Each node shows:

```
status

latency
```

```
queue

last update
```

Example:

```
SEMANTIC

HEALTHY
38 ms
Q 0
9.8 Hz
```

Status colour should be secondary.

Status text must always exist.

---

57. Queue Visualization

Instead of another chart:

```
QUEUE  ■■□□
```

or:

```
Q 2 / 4
```

Keep it simple.

---

58. Degraded State

Example:

```
SEMANTIC

DEGRADED
183 ms

FRAME DROPPING ACTIVE
```

Connection between nodes may turn amber.

59. CONFIG Screen

Do not show raw YAML as the only interface.

Left category navigation:

```
MAP

RESOLUTION

SEMANTICS

TRACKING

VEHICLE

TRAVERSABILITY

VISUALIZATION
```

Right:

structured controls.

60. Configuration Change Safety

Editing should not immediately apply every keystroke.

Workflow:

```
EDIT

→ Review Changes

→ Apply
```

Display diff:

```
max_level
3 → 2
```

```
uncertainty_threshold
0.45 → 0.52
```

---

61. Advanced YAML

Provide:

```
VIEW YAML
```

for engineering users.

Do not make it the default UI.

---

62. Command Palette

Shortcut:

```
Ctrl/Cmd + K
```

Commands:

```
Go to Mission

Switch to Resolution view

Toggle point cloud

Pause stream

Reset camera

Open cell inspector

Compare maps

Enable presentation mode
```

This makes AVR Console feel like a real engineering application.

---

63. Presentation Mode

This is highly recommended for SIH.

Shortcut:

```
P
```

Presentation Mode:

```
hides navigation rail

increases UI labels by ~10–15%

enlarges telemetry

keeps scene dominant

removes advanced debug controls
```

Ideal for projector/demo use.

---

64. Presentation Mode Compare

Recommended judge demo view:

```
UNIFORM     ◀──── wipe ────▶     AVR MAP
```

Bottom:

```
CELLS        ↓ 81%

MEMORY       ↓ 77%

UPDATE       ↓ 42%

HAZARD RETENTION  97%
```

Again: actual values only.

---

65. Typography System

Use the IBM Plex family.

IBM Plex provides Sans, Sans Condensed and Mono variants and is available under an open-font licence.

Recommended:

```
UI / BODY
IBM Plex Sans

MICRO LABELS
IBM Plex Sans Condensed

DATA / METRICS / IDs
IBM Plex Mono
```

---

66. Why IBM Plex

It communicates:

```
engineering

research

technical software

precision
```

without appearing like:

```
gaming

marketing

generic startup UI
```

The three variants also create hierarchy without mixing unrelated font families.

---

67. Typography Weights

Use:

```
400 Regular

500 Medium

600 SemiBold
```

Avoid:

```
700–900
```

except perhaps one rare emphasis.

This is an engineering console, not a landing page.

---

68. Type Scale

Recommended:

Role	Size
Tiny metadata	10px
Micro label	11px
Secondary label	12px
Body	13px
Control	13–14px
Panel heading	15–16px
Major value	20–24px
Presentation value	28–32px

---

69. Numeric Typography

Metrics MUST use:

```
font-variant-numeric: tabular-nums;
```

Numbers should not shift horizontally when updating.

Use IBM Plex Mono.

Example:

```
09.8 Hz
74.3 ms
018294 cells
```

---

70. Uppercase Rules

Uppercase only:

```
MICRO LABELS

STATUS

SECTION NAMES
```

Example:

```
GROUND
SEMANTIC
CLEARANCE
```

Do not write paragraphs in uppercase.

---

71. Spacing System

Base unit:

```
4 px
```

Scale:

```
4
8
12
16
20
24
32
40
```

This supports dense technical layouts without arbitrary spacing.

---

72. Border Radius

This UI should not look soft.

Use:

```
2px
4px
6px
```

Maximum normal radius:

```
6px
```

Avoid:

```
16px
20px
24px
full pills
```

unless a circular control genuinely requires it.

---

73. Surface Hierarchy

Use tonal separation more than shadows.

```
BACKGROUND

WORKSPACE
```

```
PANEL

CONTROL

HOVER
```

Each increases slightly in perceived lightness.

---

74. Core Neutral Palette

Recommended starting palette:

```
BG-0
#070A0D

BG-1
#0B1015

BG-2
#10171E

SURFACE
#151E27

BORDER
#1F2B35

BORDER STRONG
#30404C

TEXT PRIMARY
#E6EDF3

TEXT SECONDARY
#A5B1BA

TEXT MUTED
#6E7B85
```

---

75. Primary Interaction Colour

Use a restrained cyan:

```
SIGNAL CYAN
#55C6D4
```

Used for:

```
selected controls

focus

active view

selected geometry

links
```

Do not use it to mean:

```
safe
healthy
```

Those have separate colours.

---

76. System Status Colours

```
HEALTHY
#78C990

WARNING
#E3B866

CRITICAL
#EA6B67

UNKNOWN
#A596C8

INFORMATION
#77A9E8
```

---

77. Why Colour Roles Must Be Separate

If green means:

```
button active

AND

system healthy

AND

terrain traversable

AND

selected object
```

the interface becomes confusing.

Therefore:

```
cyan = interaction

green = safe / healthy

amber = warning / caution

red = blocked / critical

violet = unknown / unresolved
```

---

78. Use OKLCH for Design Tokens

Implement colours internally using CSS `oklch()` where practical.

OKLCH uses perceptual lightness and enables more visually consistent colour relationships than conventional HSL. MDN notes that OKLCH is widely available across modern browsers and that Oklab-based colour spaces are designed to be perceptually uniform.

Example:

```
:root {
  --bg-0: oklch(14% 0.012 245);
  --bg-1: oklch(17% 0.014 245);
  --surface: oklch(21% 0.018 245);

  --text-1: oklch(94% 0.008 235);
  --text-2: oklch(76% 0.015 235);
```

```
  --accent: oklch(78% 0.10 195);
}
```

Exact values should be visually tested.

---

79. Resolution Palette

Recommended:

```
5 cm
#62D3C6
turquoise

10 cm
#70B8E8
blue

20 cm
#D6B45F
sand/amber

40 cm
#A18AC8
violet
```

Fine → coarse should not imply:

```
good → bad
```

Therefore avoid green→red for resolution.

---

80. Semantic Palette

Suggested:

```
ROAD
#7E8B95

TERRAIN
#8DB174

VEGETATION
```

```
#4F8A6C

VEHICLE
#6FA9EC

PERSON
#ED8177

POLE / SMALL STRUCTURE
#DEB65F

BUILDING
#858DA0

UNKNOWN
#A596C8
```

Semantic palette is configurable per dataset.

---

81. Traversability Palette

Here green→amber→red is appropriate because it represents safety.

```
TRAVERSABLE
green

CAUTION
amber

BLOCKED
red

UNKNOWN
violet-grey
```

But never rely on colour alone.

---

82. Accessibility Contrast

Normal text should meet at least:

```
4.5 : 1
```

contrast.

Large text:

```
3 : 1
```

W3C also recommends at least 3:1 contrast for meaningful UI components and graphical indicators.

---

83. Do Not Encode State With Colour Alone

Example:

Bad:

```
●
```

green/red only.

Good:

```
● HEALTHY

▲ WARNING

× CRITICAL

? UNKNOWN
```

Charts should also use:

```
colour
+
dash style
+
labels
```

where needed.

---

84. Panel Styling

Panels use:

```
1 px border

opaque dark surface

small 4px radius
```

No giant shadows.

Example:

```
.panel {
  background: var(--bg-2);
  border: 1px solid var(--border);
  border-radius: 4px;
}
```

---

85. Canvas Overlays

Small overlays inside the 3D scene MAY use:

```
92–96% opaque dark surface
```

Do not use aggressive backdrop blur.

Blur is visually fashionable but adds GPU work precisely where AVR Console is already rendering heavy 3D data.

---

86. Shadows

Normal panels:

```
no shadow
```

Floating:

```
command palette
modal
context menu
```

may use a subtle shadow.

Use border and tonal contrast first.

---

87. Motion Philosophy

Animation communicates:

```
state transition

spatial relationship

interaction feedback
```

Animation does NOT exist for decoration.

---

88. Motion Library

Use:

Motion for React

only for DOM-level interface transitions.

Motion supports native-browser animation paths where possible and provides controlled layout/enter/exit animation primitives.

Do not animate Three.js data using Motion.

---

89. Motion Durations

```
hover
80–120 ms

control selection
120–160 ms

drawer
180 ms

inspector
180–220 ms

modal
```

```
180–220 ms

page transition
150–200 ms
```

---

90. Easing

Recommended:

```
cubic-bezier(0.2, 0.8, 0.2, 1)
```

Do not use:

```
bouncy springs
rubber effects
overshoot
```

in an engineering HMI.

---

91. Cell Refinement Animation

For presentation mode only:

When:

```
40cm cell
→
10cm children
```

show a subtle:

```
outline split
+
crossfade
```

around:

```
150–220 ms
```

Do not move cells through space.

The geometry must remain truthful.

---

92. Live Value Updates

Do not count numbers up theatrically.

Example:

Bad:

```
24MB
25MB
26MB
...
```

animated.

Good:

```
direct update
```

possibly with a 100ms colour acknowledgement on major state change.

---

93. Loading Animation

Avoid shimmer skeletons.

Use AVR nested-square motif.

Example:

```
▣

CONNECTING TO AVR STREAM

ROS bridge       connected

semantic         waiting

mapper           waiting
```

---

94. Reduced Motion

Respect:

```
@media (prefers-reduced-motion: reduce)
```

MDN recommends removing or reducing non-essential motion when users have requested reduced motion.

In reduced-motion mode:

```
drawers appear instantly

no cell split animation

no panel slide

no pulse effects
```

---

95. UI State — Connected

Normal:

```
● LIVE
```

No pulsing glow.

A stable connection should look calm.

---

96. UI State — Connecting

```
○ CONNECTING

Last frame —
```

Nested-square loader may animate once per second.

---

97. UI State — Stale

If no new data:

```
STALE 2.8s
```

Canvas retains last known frame but:

```
slightly dims
```

and top bar becomes amber.

---

98. UI State — Disconnected

Do NOT replace everything with a blank page.

Keep the last scene visible.

Overlay:

```
STREAM DISCONNECTED

Last valid frame
23:41:18.320

Retrying...
```

This preserves operator context.

---

99. UI State — Paused

Scene remains normal.

Top:

```
PAUSED
```

Timeline enabled.

No modal.

100. Alerts

Do not use toast spam for continuous system problems.

Use:

Toast

for:

```
snapshot saved
config exported
layout restored
```

Persistent banner

for:

```
semantic provider offline
pose unavailable
high frame loss
mapping degraded
```

101. Empty States

Example:

```
NO POINT STREAM

Waiting for /avr/points_semantic

Expected:
sensor_msgs/PointCloud2
```

Useful, technical, actionable.

Not:

```
Oops! Nothing here 🤖
```

102. Interaction Density

This UI is allowed to be dense.

But density must be structured.

Technique:

```
default → essential information

selection → contextual information

drawer → advanced information

diagnostics → engineering detail
```

This is progressive disclosure.

---

103. Tooltips

Delay:

```
300–450 ms
```

Useful for:

```
icons

abbreviations

resolution levels
```

No tooltip for obvious labeled controls.

---

104. Keyboard Interaction

Required:

```
Space
Play / pause
```

```
R
Reset camera

F
Focus selected object

L
Layers

1
Perspective

2
Top

3
Follow vehicle

C
Compare

P
Presentation mode

Cmd/Ctrl + K
Command palette

Esc
Close drawer / selection
```

---

105. Focus Style

Keyboard focus:

```
2px signal-cyan outline
+
2px offset
```

Never remove browser focus without replacing it.

---

106. Accessible UI Primitives

Use:

Radix Primitives

for:

```
dialogs
dropdowns
tabs
tooltips
switches
selects
menus
sliders
```

Radix is deliberately unstyled and handles much of the WAI-ARIA/focus/keyboard behaviour, allowing AVR Console to keep a custom visual language instead of inheriting a component-library appearance.

---

107. Very Important UI Library Decision

Do NOT build AVR Console using a default:

```
shadcn dashboard template
```

Shadcn is technically good, but AI coding tools often reproduce the same:

```
sidebar
rounded cards
big statistic tiles
grey dashboard
```

look.

Use:

```
Radix primitives
+
custom components
+
custom CSS tokens
```

instead.

---

108. CSS Strategy

Recommended:

```
CSS Modules

+

global design-token CSS
```

Structure:

```
styles/
│
├── tokens.css
├── reset.css
├── typography.css
├── theme.css
├── utilities.css
└── motion.css
```

Components own their module styles.

---

109. Tailwind Policy

Tailwind MAY be used only as a layout/utility accelerator.

If Antigravity uses Tailwind:

do NOT allow the UI to become:

```
rounded-xl
rounded-2xl
shadow-lg
bg-gradient-to-r
backdrop-blur-xl
```

by habit.

All styling must follow this specification.

---

110. Frontend Framework

Recommended:

```
React
TypeScript
Vite
```

Vite provides fast development/HMR and optimized production builds, making it suitable for a local engineering dashboard.

No SSR is required.

Therefore Next.js adds unnecessary complexity.

---

111. Recommended Frontend Stack

```
React
TypeScript
Vite

Three.js
Radix Primitives
Zustand
Motion
uPlot

CSS Modules
PostCSS

Vitest
Playwright
axe-core
```

---

112. Why Not Next.js

AVR Console is:

```
local / workstation

real-time
```

```
WebSocket-driven

3D-rendering-heavy
```

It does not need:

```
SEO

SSR

server components

public content routing
```

Therefore Vite is simpler.

---

113. State Management

Use:

Zustand

for:

```
UI state

current view

selected cell

selected track

low-frequency metrics

connection state

config state
```

Do NOT store full point clouds or every map cell inside React state.

---

114. High-Frequency Data Path

Bad:

```
WebSocket
→ setState(points)
→ React rerender
→ 100k objects
```

Correct:

```
WebSocket
→ Worker
→ ArrayBuffer
→ renderer ref
→ GPU BufferAttribute
```

---

115. Web Worker

Use a worker for:

```
binary packet decoding

semantic palette conversion where needed

buffer preparation

heavy filtering
```

Keep DOM thread responsive.

---

116. OffscreenCanvas

OffscreenCanvas may later move rendering work into a worker and is widely available in modern browsers.

However it complicates event handling and Three.js architecture.

Therefore:

```
V1
main-thread WebGL rendering
+
worker decoding
```

```
V2
evaluate OffscreenCanvas
```

---

117. Data Transport

Do not stream high-volume point clouds as huge JSON arrays.

Use:

```
WebSocket
+
binary ArrayBuffer
```

for:

```
point cloud

adaptive cells
```

Use JSON for low-volume:

```
metrics

status

config

tracks if count is small
```

---

118. Frontend Data Adapter

Create:

```ts
interface AVRDataSource {
  connect(): Promise<void>
  disconnect(): void

  onFrame(handler): void
  onMap(handler): void
  onTracks(handler): void
  onMetrics(handler): void
```

```
  onStatus(handler): void
}
```

Implement:

```
MockDataSource

WebSocketDataSource
```

---

119. Why MockDataSource Matters

Antigravity can build the full frontend before the actual ROS bridge is finished.

The UI development should not block on robotics backend integration.

Mock source should simulate:

```
point cloud

adaptive cells

moving object

metrics

resolution changes
```

---

120. Mock Scene

Create one deterministic demo scene:

```
road

vehicle

pedestrian

pole

rough terrain

overhead obstacle
```

```
coarse distant terrain
```

This allows consistent UI development.

---

121. Point Cloud Binary Format

Conceptually:

```
HEADER

timestamp
point_count
flags
```

followed by:

```
Float32Array positions

Uint16Array labels

Uint8Array confidence
```

Avoid nested JS objects.

---

122. Map Cell Binary Format

Prefer Struct-of-Arrays:

```
x[]
y[]
z[]
size[]
semantic[]
confidence[]
traversability[]
flags[]
```

rather than:

```
[
 {x,y,z,size,...},
```

```
 {x,y,z,size,...},
 ...
]
```

for better memory behaviour.

---

123. Renderer Grouping

Group cells by:

```
resolution
```

and potentially:

```
visibility mode
```

rather than material-per-cell.

Target low draw-call count.

---

124. Scene Labels

DOM labels are expensive.

Display persistent labels only for:

```
selected object

high-risk object

nearest few tracked objects
```

Everything else can remain geometry.

---

125. Rendering Optimizations

Use:

```
BufferGeometry
```

```
InstancedMesh

typed arrays

frustum culling

reused buffers

limited DPR

low material count

low draw-call count
```

Never allocate new point objects every render frame.

---

126. Hidden Tab Behaviour

When:

```
document.hidden = true
```

reduce or pause expensive rendering.

Data connection may continue.

---

127. Render-On-Demand

When playback is paused and camera is stationary:

```
do not continuously redraw at 60fps
```

Render only when:

```
camera changes
selection changes
UI changes
```

---

128. Performance Targets

Desktop reference target:

```
3D renderer
55–60 FPS

data updates
10 Hz

UI interactions
<100 ms perceived response

main-thread long tasks
avoid >50 ms
```

---

129. Renderer Performance Modes

Provide:

```
QUALITY

BALANCED

PERFORMANCE
```

They affect:

```
point size

point count/downsampling

DPR

visualization update frequency

labels
```

They must NOT alter backend mapping results.

Only visualization complexity changes.

---

130. Frontend Folder Structure

```
frontend/
│
├── src/
│   ├── app/
│   │
│   ├── routes/
│   │   ├── mission/
│   │   ├── compare/
│   │   ├── benchmarks/
│   │   ├── diagnostics/
│   │   └── config/
│   │
│   ├── components/
│   │   ├── shell/
│   │   ├── controls/
│   │   ├── inspectors/
│   │   ├── telemetry/
│   │   └── charts/
│   │
│   ├── scene/
│   │   ├── AVRScene.ts
│   │   ├── PointCloudRenderer.ts
│   │   ├── AdaptiveGridRenderer.ts
│   │   ├── TrackRenderer.ts
│   │   ├── VehicleRenderer.ts
│   │   ├── palettes.ts
│   │   └── shaders/
│   │
│   ├── data/
│   │   ├── AVRDataSource.ts
│   │   ├── MockDataSource.ts
│   │   ├── WebSocketDataSource.ts
│   │   ├── decoder.worker.ts
│   │   └── protocol.ts
│   │
│   ├── stores/
│   │
│   ├── hooks/
│   │
│   ├── styles/
│   │
│   └── utils/
│
├── public/
├── tests/
└── vite.config.ts
```

---

131. Component Inventory

Core components:

```
AppShell

TopSystemBar

NavRail

MissionViewport

ViewportToolbar

LayerDrawer

CameraControls

ResolutionLegend

TelemetryStrip

PlaybackTimeline

CellInspector

TrackInspector

CompareViewport

CompareMetrics

MetricPlot

ExperimentSelector

PipelineDiagnostics

PipelineNode

ConfigPanel

CommandPalette

SystemAlert

ConnectionOverlay
```

---

132. Component Rule

Every component should answer:

```
What information does this provide?

What action does it allow?

Would removing it make the user less informed?
```

If the answer is:

```
it looks cool
```

remove it.

---

133. Canvas Accessibility

Canvas/WebGL graphics are not inherently understandable to assistive technology.

MDN notes that canvas content itself does not automatically expose the meaning of drawn objects.

Therefore selected scene data must also exist in DOM form.

Examples:

```
selected cell inspector

track table

system metrics

status text
```

---

134. Accessibility Testing

Use:

```
axe-core

Playwright keyboard tests
```

Verify:

```
keyboard navigation

focus order

contrast

dialog focus management

reduced motion

colour-independent status
```

---

135. Error UX

Errors should answer:

```
WHAT FAILED?

WHAT IS AFFECTED?

IS SYSTEM STILL RUNNING?

WHAT CAN I DO?
```

Bad:

```
Something went wrong.
```

Good:

```
SEMANTIC STREAM LOST

Adaptive geometry mapping is still active.

Last semantic frame: 2.4s ago.
```

```
Retrying automatically.
```

---

136. Metrics Language

Prefer:

```
Map update

End-to-end

Active cells

Map memory

Dropped frames

Track count

Semantic latency
```

Avoid unnecessary abbreviations.

Abbreviations permitted where standard:

```
FPS

p95

Hz

RAM
```

---

137. Data Truth Principle

Never animate or smooth data in a way that implies measurements that were not observed.

Track movement MAY interpolate visually between two known positions.

But timestamps and actual values remain visible.

---

138. No Fake Demo Numbers

Mock mode should show:

```
DEMO DATA
```

clearly.

When connected to the backend:

```
LIVE
```

The UI must never accidentally present random generated benchmark numbers as experimental results.

---

139. SIH Demo Flow

Recommended presentation sequence:

```
1.
MISSION
show semantic LiDAR

2.
switch to RESOLUTION
explain 5/10/20/40 cm

3.
click distant pole
show "WHY THIS RESOLUTION?"

4.
open COMPARE
wipe uniform vs adaptive

5.
show live cell/memory difference

6.
show moving object track

7.
switch TRAVERSABILITY

8.
```

```
show overhang clearance

9.
open BENCHMARKS
show measured experiments

10.
DIAGNOSTICS briefly
show complete pipeline running in real time
```

The frontend should make this flow nearly effortless.

---

140. Visual Detail That Will Make It Feel Custom

Around the 3D viewport use subtle metric rulers:

```
20m   40m   60m
```

and camera scale.

Selected cells display:

```
L1 / 10cm
```

in a small technical callout.

Top-left scene identifier:

```
ODOM FRAME
X +18.23
Y -4.11
```

These details make the product feel like real spatial-analysis software rather than a template.

---

141. Resolution Reason Tags

Avoid pills.

Use compact rectangular technical tags:

```
SEM  POLE
```

```
UNC  0.09

DYN  NO

DEN  PASS
```

Possible style:

```
┌─────────────┐
│ SEM │ POLE  │
└─────────────┘
```

---

142. Borders and Section Headers

Panel header:

```
RESOLUTION
──────────────────
```

or:

```
RESOLUTION ──────────
```

This scientific-document style is preferable to giant headings.

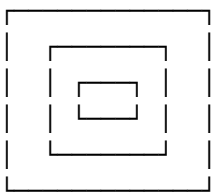
---

143. Unique Empty Background Motif

When no stream exists, use a very faint nested-grid illustration.

No random 3D animation.

Example:



```
      WAITING FOR MAP
```

---

144. Anti-AI-Generated UI Rules for Antigravity

These requirements are mandatory when prompting the coding agent:

```
DO NOT use a generic SaaS dashboard template.

DO NOT create statistic cards across the top.

DO NOT use gradient blobs.

DO NOT use glassmorphism throughout.

DO NOT use oversized rounded cards.

DO NOT use huge 40–60px headings.

DO NOT use random decorative charts.

DO NOT use excessive badges or pills.

DO NOT use purple as the primary "AI" colour.

DO NOT use fake values unless explicitly marked MOCK.

DO NOT make every hover state glow.

DO NOT use generic hero sections.

DO NOT create a login screen.

DO NOT place an avatar/profile menu unless actually needed.

DO NOT use a marketing landing page before the console.
```

---

145. What Antigravity SHOULD Do

```
Build the 3D scene first.

Treat the viewer as the primary UI.

Use narrow technical navigation.
```

```
Use custom design tokens.

Use IBM Plex.

Use small radii.

Use subtle borders.

Use Radix primitives without default component-library styling.

Use true map cells and actual resolution sizes.

Use mono numeric typography.

Build explainability directly into the cell inspector.

Keep live metrics compact.

Prioritize performance over decorative motion.
```

---

146. Implementation Order for Antigravity

Stage 1 — Design system

Build:

```
tokens
fonts
typography
buttons
tabs
menus
panel
navigation
status
```

No dashboard yet.

Stage 2 — App shell

```
top bar
nav rail
viewer region
```

```
telemetry
inspector
```

Stage 3 — Mock 3D scene

Implement:

```
point cloud
vehicle
adaptive cells
camera
resolution colours
```

Stage 4 — Mission UX

```
view modes
layers
selection
cell inspector
tracks
timeline
```

Stage 5 — Compare

```
synchronized views
wipe
metrics
```

Stage 6 — Benchmarks

```
uPlot
experiments
tables
```

Stage 7 — Diagnostics

```
pipeline diagram
node health
```

Stage 8 — Backend integration

Swap:

```
MockDataSource
→
WebSocketDataSource
```

Stage 9 — Accessibility & performance

Profile and polish.

---

147. Definition of Frontend Completion

The frontend is considered complete when:

```
application opens directly to Mission

3D LiDAR scene is smooth

adaptive cells visibly differ in scale

six colour modes work

cell selection explains resolution

dynamic objects can be inspected

uniform/adaptive comparison works

benchmark charts consume real data

pipeline diagnostics display backend health

keyboard shortcuts work

presentation mode works

1366×768 is usable

no generic template UI remains

mock and live data are clearly distinguished

UI remains responsive during a 10Hz data stream
```

---

148. Final Design Stack

```
FRAMEWORK
React + TypeScript + Vite

3D
Three.js
BufferGeometry
Points
InstancedMesh

STATE
Zustand

ACCESSIBLE PRIMITIVES
Radix UI

ANIMATION
Motion for React

CHARTS
uPlot

STYLING
CSS Modules
CSS Variables
OKLCH design tokens

TYPOGRAPHY
IBM Plex Sans
IBM Plex Sans Condensed
IBM Plex Mono

ICONS
Phosphor
+
custom AVR spatial icons

TRANSPORT
WebSocket
binary ArrayBuffers

WORKERS
Web Worker decoding

TESTING
Vitest
Playwright
axe-core
```

149. Final Visual Identity

AVR Console should feel like:

> **A precision scientific instrument built specifically for understanding an adaptive LiDAR map.**

Not like:

> **A website displaying robotics data.**

The distinction matters.

The final experience should be:

```
scene-first

quiet

dark

precise

responsive

technical

explainable

information-dense

visually disciplined
```

with AVR-Map's unique concept—**adaptive spatial resolution**—embedded directly into the visual language of the interface.

The UI itself should therefore communicate the central project idea:

> **Detail appears where information matters.**